东 南 大 学 考 试 卷（A 卷）

课程名称 算法设计与分析 考试学期 2017-2018-2 得分 ____

适用专业 计算机 考试形式 开卷 考试时间长度 150 分钟

（可携带纸质教材、课件、讲义、笔记）

姓名 线 学号 封 密

1. 算法 A 的时间复杂度为$T_A = O(n\log n)$，那么用 $O(\mathrm{n}^2\log n)$表示算法 A 的时间复杂度（即$T_A = O(\mathrm{n}^2\log n)$）的写法正确吗？（7 分）

- **正确性：** $T_A = O(n^2\log n)$ **数学上是正确的。**
- **推荐写法：** 应写成 $T_A = O(n\log n)$，因为这是时间复杂度的最紧确表示。
- **规范：** 时间复杂度表示遵循“尽量紧确”的原则，避免用过于宽松的上界描述。

2. 已知一个 NP 完全问题 A 和一个 NP 问题 B，假设现已证明 A 可多项式时间规约到 B 问题的一个实例，那么 B 可多项式时间规约到 A 的一个实例吗?为什么？（10 分）

不一定。原因是多项式时间规约的方向性取决于问题之间的关系，而不是对称的。我们需要具体分析 A 和 B 的性质

1. NP完全性与多项式时间规约的定义

B：表示可以通过一个多项式时间的算法，将 A 的任意实例 x 转化为 B 的一个实例 y，使得 x 的解可以从 y 的解中直接获得。这表明：**如果 B 可在多项式时间内求解，则 A 也可在多项式时间内求解。**

- NP完全问题的定义：
  - A 是 NP问题。
  - 所有 NP问题都可多项式时间规约到 A。

2. 已知 $A \leq_p B$ 的含义

$A \leq_p B$，这说明**解决 B 至少不比解决 A 更困难。**

- 而 B 仅是 NP问题，并不一定是 NP完全问题。因此，没有保证所有 NP问题都可以规约到 B，特别是没有保证 $B \leq_p A$。

3. 为什么 B 不一定可规约到 A?

- 如果 B 是一个 **NP完全问题**，则可以证明所有 NP问题 (包括 A) 都可以规约到 B，因此 $B \leq_p A$ 是成立的。
- 但如果 B 只是一个普通的 NP问题，则我们无法保证 B 能规约到任何 NP完全问题 (包括 A)。此时，B 可能比 A 更简单，或者与 A 无直接联系。

4. 举例说明

- A 是 SAT (布尔可满足性问题)，一个典型的 NP完全问题。
- B 是某个特殊的 NP问题，例如一个只对部分特定输入有解的 SAT 子问题 (如某些变量数量受到限制)。
  - 我们可以通过规约 $A \leq_p B$，表明如果能够解决 B，就能解决 A。
  - 但反过来，B 不一定能规约到 A，因为 B 可能比 A 更简单，甚至比 A 的某些约束更强。

5. 结论

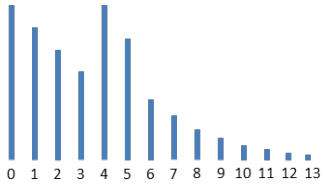
- 如果 B 是 **NP完全问题**，那么 $B \leq_p A$ 必定成立。
- 如果 B 是一个普通的 **NP问题**，则 $B \leq_p A$ 不一定成立。
- 因此，问题的答案是：**不一定。**

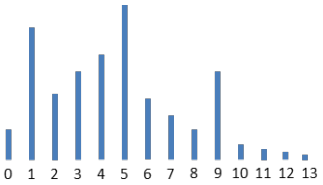
3. 对于有些问题为何需要设计近似算法求解。（8 分）

见 2016 卷

4. 在建滑雪道时，需要铲掉道上凸起小丘以形成坡道，一条坡道的高度递减或递增。现考虑简化的一维滑雪道问题。给定n个整数 $a[0], a[1], \ldots, a[n-1]$，分别表示雪道上的高度，请设计一种算法，降

低最少的整数，使之成为只有一条或只有两条坡道的滑雪道。如下左图所示，降低 2 个整数（将下标为 4 处和 5 处的高度降低到比 3 处低，比 6 处高，且 4 处比 5 处高），则可形成一条滑雪坡道。又如下右图所示，降低 2 个整数（将 1 处的高度降低到比 0 处高，比 2 处低；将 9 处的高度降低到比 8 处低，比 10 高），则可形成两条滑雪坡道（15 分）





解决方案

1. 单调递增或递减坡道

使用动态规划解决。假设 dp[i] 表示将数组的前 i 个元素调整为单调递增所需降低的最小高度：

- 如果数组本身是递增的，保持原样无需调整。
- 如果某个元素大于前一个元素，则需要降低高度。

类似地，也可以计算数组调整为**单调递减**所需的最小高度。

2. 两条坡道的情况

将问题划分为两个子问题：

- 在位置 k 分割数组，前 k 个调整为单调递增，后 $n-k$ 个调整为单调递减。
- 目标是找到一个 k，使得所需降低的总高度最小。

我们可以枚举分割点 k，并结合动态规划的结果求解。

算法设计

1. 定义状态

- dp_inc[i]：调整前 i 个元素为单调递增所需降低的最小高度。
- dp_dec[i]：调整前 i 个元素为单调递减所需降低的最小高度。

2. 状态转移

- **单调递增：**

$$dp_inc[i] = dp_inc[i-1] + \max(0, a[i-1] - a[i])$$

如果当前高度 $a[i-1] > a[i]$，需要降低当前高度 $a[i-1] - a[i]$。

- **单调递减：**

$$dp_dec[i] = dp_dec[i-1] + \max(0, a[i] - a[i-1])$$

3. 两条坡道的组合

- 枚举分割点 k，计算：

$$cost = dp_inc[k] + dp_dec[n-1] - dp_dec[k]$$

表示将前 k 个元素调整为单调递增，后 $n-k$ 个调整为单调递减的总代价。

```python
def min_reduction(a):
    n = len(a)
    dp_inc = [0] * n  # 前i个调整为单调递增的代价
    dp_dec = [0] * n  # 前i个调整为单调递减的代价

    # 计算单调递增的dp
    for i in range(1, n):
        dp_inc[i] = dp_inc[i-1] + max(0, a[i-1] - a[i])

    # 计算单调递减的dp
    for i in range(1, n):
        dp_dec[i] = dp_dec[i-1] + max(0, a[i] - a[i-1])

    # 单调递增/递减的最小代价
    min_cost_single = min(dp_inc[-1], dp_dec[-1])

    # 两条坡道的最小代价
    min_cost_two = float('inf')
    for k in range(1, n):
        min_cost_two = min(min_cost_two, dp_inc[k] + (dp_dec[-1] - dp_dec[k]))

    # 返回结果
    return min(min_cost_single, min_cost_two)
```

5. 给定一个由n个整数构成的集合S和另一个整数x，判断 S 中是否有两个元素的和为x。试设计一个时间复杂度为 $O(n\log n)$的算法求解上述问题。（15 分）

算法设计

1. **排序**：
   - 将集合 S 排序。排序的时间复杂度为 $O(n \log n)$。
2. **双指针法**：
   - 在排序后的数组上使用双指针技巧：
     - 一个指针从数组的最左端（较小数）开始。
     - 另一个指针从数组的最右端（较大数）开始。
   - 通过指针的移动，寻找两个数之和是否等于 x。
     - 如果当前和小于 x，则左指针右移（增加和的大小）。
     - 如果当前和大于 x，则右指针左移（减小和的大小）。
     - 如果当前和等于 x，则找到答案。

```python
def has_pair_with_sum(S, x):
    # 1. 对集合进行排序
    S.sort()  # 时间复杂度 O(n log n)

    # 2. 初始化双指针
    left = 0
    right = len(S) - 1

    # 3. 双指针搜索
    while left < right:
        current_sum = S[left] + S[right]

        if current_sum == x:  # 找到满足条件的两个数
            return True
        elif current_sum < x:  # 当前和小于目标值，左指针右移
            left += 1
        else:  # 当前和大于目标值，右指针左移
            right -= 1

    # 没有找到满足条件的数对
    return False
```

6. 对一棵树进行着色，每个结点可着黑色或白色，相邻结点不能着相同颜色。令树的根为r，请设计一种算法对树中尽量多的节点着黑色。（15 分）

算法思路

1. **状态定义**：
   - 定义 $dp[u][0]$ 为以节点 u 为根的子树中，当节点 u 着白色时，子树中最多的黑色节点数。
   - 定义 $dp[u][1]$ 为以节点 u 为根的子树中，当节点 u 着黑色时，子树中最多的黑色节点数。
2. **状态转移**：
   - 如果当前节点 u 着白色（$dp[u][0]$）：

$$dp[u][0] = \sum_{v \in \text{children of } u} \max(dp[v][0], dp[v][1])$$

意味着子节点 v 可以自由选择是白色还是黑色。

   - 如果当前节点 u 着黑色（$dp[u][1]$）：

$$dp[u][1] = 1 + \sum_{v \in \text{children of } u} dp[v][0]$$

意味着子节点 v 必须着白色。

3. **初始化**：
   - 如果节点是叶子节点：
     - $dp[u][0] = 0$（叶子节点着白色时没有黑色节点）。
     - $dp[u][1] = 1$（叶子节点着黑色时本节点就是黑色）。
4. **目标**：
   - 对根节点 r，取：

$$\max(dp[r][0], dp[r][1])$$

5. **优化子结构性质**：
   - 子树的最优解独立于其他子树，只与父节点的状态有关，具备**优化子结构**性质。
6. **子问题重叠性质**：
   - 各个子树间的子问题会递归计算，适合动态规划。

```python
def max_black_nodes(tree, root):
    # 构建树的子节点关系
    children = build_children(tree)

    # 动态规划函数
    def dfs(u):
        dp0, dp1 = 0, 1  # 初始化：dp[u][0], dp[u][1]
        for v in children[u]:
            dp_child0, dp_child1 = dfs(v)
            dp0 += max(dp_child0, dp_child1)
            dp1 += dp_child0
        return dp0, dp1

    dp_root0, dp_root1 = dfs(root)
    return max(dp_root0, dp_root1)

# Helper function to build the tree
def build_children(tree):
    children = {node: [] for node in tree}
    for parent, child in tree.items():
        children[parent].append(child)
    return children
```

7. 某公司有个工厂和仓库。由于原材料等价格波动，工厂每个月的生产成本也会波动，令第i个月产品的单位生产成本为c_i（该月生产一个产品的成本为c_i）。仓库储存产品的成本也会波动，令第i个月产品的单位储存成本为s_i（该月存储一个产品的成本为c_i）。令第i个月需要供应给客户的产品数量为y_i，仓库里的和生产的产品均可供应给客户。假设仓库的容量无限大，供应给客户剩余的产品可储存在仓库中。若已知n个月中各月的单位生产成本c_i、单位储存成本s_i，以及产品供应量y_i，请设计一时间复杂度为$O(n)$算法求这n个月的最低总成本。（15 分）

算法设计思路

1. **问题抽象：**
   - 每个月有两种决策：
     1. 直接生产满足当月需求 y_i，成本为 $c_i \cdot y_i$。
     2. 从之前的某个月 j 生产的多余产品储存到本月使用，成本为 $c_j \cdot y_i + \sum_{k=j}^{i-1} s_k \cdot y_i$。
2. **贪心策略：**
   - 在满足每个月需求 y_i 的时候，优先从较早的月份生产，因为生产成本 c_j 是固定的，累积存储成本 $\sum_{k=j}^{i-1} s_k$ 随 j 增大而增加。我们只需要从过去的月份中找到满足最低总成本的月 j。
   - 当前月 i 的需求 y_i 由过去尽可能低成本的生产供应，不需要回溯。
3. **状态变量：**
   - $\text{remain}[j]$：表示第 j 个月生产的剩余产品数量，可以供未来使用。
   - 当前月 i 的成本由两个部分构成：
     - 从某个月生产供应的成本。
     - 剩余产品存储到本月的存储成本。
4. **动态维护：**
   - 每个月我们计算将当前需求 y_i 分配到之前生产的产品上，按贪心策略从总成本最低的月份 j 开始分配。
   - 记录剩余产品数量和累积成本即可。

```python
def min_total_cost(n, c, s, y):
    # 初始化总成本
    total_cost = 0

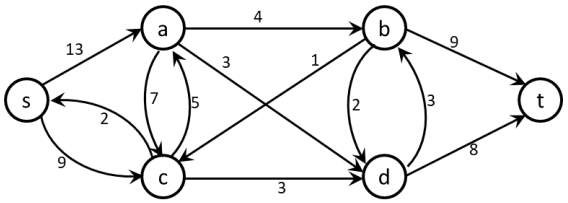
    # 初始化上个月的剩余产品成本
    remain = 0  # 仓库中剩余的产品数量
    min_cost = c[0]  # 当前最低的生产成本

    # 遍历每个月，逐步分配需求
    for i in range(n):
        # 更新当前的最低生产成本（考虑储存成本）
        min_cost = min(min_cost + s[i], c[i])

        # 当前月的成本 = 最优生产成本 x 当前月需求
        total_cost += min_cost * y[i]

    return total_cost
```

8. 下图是一个流网络。请计算 *s* 到 *t* 的最大流值是多少？并标示出一种流方式。（15 分）



10